



UNIVERSIDAD
DE MÁLAGA

| uma.es

Comparativa: RAG y modelos Gemini

Capa semántica de WatchGate — resultados medidos contra la API real

Equipo WatchGate — Línea 2 (reputación y semántica)

Javier Martín Jurado

Málaga, agosto de 2026

Índice

Resumen ejecutivo	2
1. Metodología	2
1.1 Casos de prueba	2
1.2 Casos difíciles (diseñados para estresar el RAG)	3
2. Comparativa con RAG vs. sin RAG (gemini-2.5-flash)	4
2.1 Lectura de los resultados	4
2.2 Casos difíciles: con RAG vs. sin RAG	5
3. Comparativa entre modelos	6
3.1 Calidad — caso trojan-source (con RAG)	6
3.2 Calibración — caso benign-plugin-loader (con y sin RAG)	6
3.3 Coste real por análisis	7
4. Sobre el catálogo de modelos	7
5. Conclusiones y recomendación	7

Resumen ejecutivo

Este informe recoge dos comparativas medidas contra la API real de Gemini (no simuladas ni estimadas), sobre la capa semántica de WatchGate:

- 1. **Con RAG vs. sin RAG** — 16 casos de prueba distintos (código real, algunos maliciosos y otros deliberadamente benignos-pero-técnicos), cada uno analizado dos veces con gemini-2.5-flash: una con el contexto RAG activado, otra con él desactivado. Los primeros 11 son casos “canónicos” (patrones muy conocidos); los otros 5 (sección 2.2) se diseñaron a propósito para estresar al RAG con casos que un LLM no puede resolver solo con su conocimiento general.
- 2. **Comparativa entre modelos** — los mismos casos más representativos, repetidos contra gemini-2.5-flash, gemini-3.5-flash y gemini-3.6-flash, con coste real calculado a partir del consumo de tokens devuelto por la propia API y el precio oficial publicado por Google.

Conclusión en una frase: en los 11 casos canónicos el RAG aporta poco (crítico en 1, indiferente en 6); pero en los 5 casos diseñados para estresarlo, el RAG **decide el veredicto en 2 de 5** (uno detecta un ataque real que pasaba desapercibido, otro evita un falso positivo grave) y revela un riesgo real en un tercero (un caso adversarial que imita a propósito un ejemplo de calibración del propio corpus). El modelo más nuevo probado (gemini-3.6-flash) da mejor calibración a menor coste que la generación intermedia (gemini-3.5-flash), aunque sigue siendo más caro que el modelo actual en producción (gemini-2.5-flash).

1. Metodología

Todas las pruebas se han ejecutado con scripts/rag_ablation_cases.py (generador de casos reutilizable, cada uno un repositorio git real de dos commits) y scripts/smoke_test_linea2.py / una variante programática del mismo flujo, llamando a SemanticLayer.analyze() de verdad – no una simulación del prompt. La tool de VirusTotal se mantuvo desactivada durante toda la comparativa, para aislar el efecto del RAG y del modelo del de esa señal adicional.

Para desactivar el RAG se apunta rag_index_path a un índice vacío (mismo mecanismo que retrieve_relevant_context ya usa cuando no existe .watchgate/rag_index): retrieve_relevant_context devuelve [] y el prompt de sistema queda sin la sección de “casos de ataque conocidos”.

1.1 Casos de prueba

Caso	Qué prueba	¿Es malicioso?
trojan-source	Caracteres Unicode bidireccionales reales insertados en un comentario de Python	Ambiguo a propósito – el “ataque” está inerte en este caso concreto
base64-exec	<i>Reverse shell</i> codificado en base64 y ejecutado con <code>exec()</code>	Sí, inequívoco
benign-plugin-loader	Carga dinámica de plugins con <code>importlib/pkgutil</code>	No – caso de calibración
dependency-confusion	Nueva dependencia con nombre de convención interna y versión inflada	Sí
docker-base-image-swap	Imagen base pinneada sustituida por un tag mutable + <code>RUN curl \ sh</code>	Sí
new-file-minimal-wiring	Payload repartido entre un fichero nuevo y una línea de enganche en uno existente	Sí
github-actions-secret-leak	Workflow de CI modificado para exfiltrar <code>secrets.*</code>	Sí
telegram-c2-exfiltration	Busca <code>.env</code> /credenciales AWS locales y las sube a un bot de Telegram	Sí
docker-entrypoint-hijack	ENTRYPOINT reescrito para descargar y ejecutar un script antes de arrancar la app	Sí
typosquat-popular-package	Nueva dependencia que imita a un paquete público real (caso <code>sympy-dev</code>)	Sí
benign-curl-checksum-verified	Usa <code>curl</code> (señal habitual de sospecha) pero con verificación de checksum	No – caso de calibración

1.2 Casos difíciles (diseñados para estresar el RAG)

Los 11 casos de la tabla anterior son patrones muy conocidos (`curl \|| bash`, `base64+exec`, *typosquatting*...) que un LLM moderno ya reconoce de su entrenamiento general, con o sin RAG. Para saber si el RAG aporta algo más allá de eso, se construyeron 5 casos nuevos, cada uno apuntando a una razón concreta por la que el RAG *debería* marcar diferencia:

Caso	Por qué es más difícil para el RAG	¿Malicioso?
npm-postinstall-worm	Patrón real de septiembre-noviembre 2025 (gusano “Shai-Hulud” en npm), posterior al corte de entrenamiento de los tres modelos probados – el modelo no puede “conocer” el incidente, solo razonar sobre el código en sí	Sí
maintainer-handoff-single-commit	Generaliza el patrón <i>estructural</i> de <code>event-stream/coa+rc/colors+faker</code> (cesión de mantenimiento + primer commit + dependencia nueva sin historial) a un paquete e incidente que no existen tal cual en el corpus	Sí
build-macro-tampering	Generaliza el patrón de <code>xz-utils</code> (payload oculto en un fixture de test, activado condicionalmente durante el build) a un mecanismo de ofuscación y un proyecto distintos	Sí
sandboxed-eval-plugin-system	Usa <code>eval()</code> – señal casi siempre sospechosa – pero de forma genuinamente segura (<code>__builtins__</code> vacío, solo funciones matemáticas permitidas)	No – calibración

Caso	Por qué es más difícil para el RAG	¿Malicioso?
fake-checksum-curl-exec	Imita a propósito la forma superficial del caso de calibración benign-curl-checksum-verified (curl + sha256sum), pero el checksum se sirve del mismo dominio no confiable que el binario – no ancla nada	Sí (mimetiza al caso benigno)

Corpus añadido para dar soporte a estos casos: shai_hulud_npm_worm_2025.md (nuevo documento, ver más abajo). Los otros cuatro casos deliberadamente **no** tienen un documento de corpus que los describa literalmente – lo que se mide es si el RAG generaliza desde casos relacionados pero distintos, no si memoriza el caso exacto.

2. Comparativa con RAG vs. sin RAG (gemini-2.5-flash)

Caso	risk_score con RAG	risk_score sin RAG	Categoría (ambos casos)	¿Cambia algo?
trojan-source	80	45	ofuscacion en ambos	RAG sube la severidad, categoría correcta en los dos
base64-exec	99	99	backdoor	Sin cambio
benign-plugin-loader	10	80	ninguna → backdoor	RAG crítico: evita un falso positivo grave
dependency-confusion	75	75	backdoor	Sin cambio
docker-base-image-swap	80	80	backdoor	Sin cambio
new-file-minimal-wiring	97	97	backdoor	Sin cambio
github-actions-secret-leak	90	99	exfiltracion	Variación menor
telegram-c2-exfiltration	95	90	exfiltracion	Variación menor
docker-entrypoint-hijack	90	95	backdoor	Variación menor
typosquat-popular-package	90	90	backdoor	Sin cambio
benign-curl-checksum-verified	80	80	backdoor (falso positivo en los dos)	El RAG no ayuda aquí

2.1 Lectura de los resultados

El RAG no aporta lo mismo en todos los casos. De los 11, en **6 no cambia nada** (el modelo ya acierta la categoría con o sin contexto), en **3 hay variación menor** ($\pm 5-9$ puntos, sin cambiar el veredicto), en **1 es la diferencia entre acertar y un falso positivo grave** (benign-plugin-loader), y en **1 no ayuda en absoluto pese a ser exactamente el tipo de caso donde debería** (benign-curl-checksum-verified).

El caso benign-curl-checksum-verified merece una nota aparte. Se diseñó como calibración (usa curl, señal habitual de sospecha, pero de forma legítima: descarga una release pinneada por versión y verifica su checksum antes de usarla). El modelo lo marca como backdoor (riesgo 80) con y sin RAG. Su propia justificación es reveladora:

“...lo que permite la introducción de código arbitrario al sistema a través de un canal externo no verificado, incluso con una suma de verificación que podría haber sido comprometida en la fuente.”

No es un razonamiento absurdo – es una postura de seguridad más estricta de la que se anticipó al diseñar el caso (un *checksum* hardcodeado sin verificación independiente de la fuente es, en efecto, una cadena de confianza con un eslabón débil). Es un hallazgo honesto, no un fallo del sistema: o bien el modelo es más cauto de lo esperado, o bien el caso de prueba no era tan inequívocamente benigno como se pensaba.

El caso trojan-source ilustra el efecto ya documentado en docs/rag_ablation_benchmark.md: el RAG puede inflar la severidad al reconocer el *nombre* de una técnica conocida. Tras el ajuste aplicado al propio documento del corpus (explicar la regla técnica exacta – en Python, todo lo posterior a # en la misma línea es inerte –, no solo un recordatorio genérico en el prompt), la diferencia (con RAG menos sin RAG) se redujo de 53 puntos (98 frente a 45, antes del ajuste) a 35 puntos (80 frente a 45, después): mejor, pero no elimina la diferencia por completo.

2.2 Casos difíciles: con RAG vs. sin RAG

Caso	risk_score con RAG	risk_score sin RAG	Categoría	¿Cambia el veredicto?
npm-postinstall-worm	98	99	backdoor en ambos	No – el payload es demasiado explícito para necesitar RAG
maintainer-handoff-single-commit	70 (backdoor)	25 (ninguna)	Cambia de categoría	Sí, decisivo: el RAG detecta el patrón, sin él pasa desapercibido
build-macro-tampering	98	95	backdoor en ambos	No – el payload es demasiado explícito
sandboxed-eval-plugin-system	5 (ninguna)	60 (backdoor)	Cambia de categoría	Sí, decisivo: el RAG evita un falso positivo grave
fake-checksum-curl-exec	70 (backdoor, confianza media)	80 (backdoor, confianza alta)	backdoor en ambos	Ambos aciertan, pero ver nota de riesgo abajo

Esto es lo que veníamos buscando. En los 11 casos canónicos el RAG cambiaba el veredicto en 1 de 11. Aquí lo cambia en **2 de 5** – porque estos casos se eligieron específicamente para no depender de conocimiento genérico ya interiorizado por el modelo.

maintainer-handoff-single-commit es el hallazgo más importante de todo el informe. El diff es mínimo: una sola línea añadiendo "term-pad-fmt": "^0.0.1" a package.json, sin ningún código ejecutable visible. Sin RAG, el modelo lo despacha como ninguna (25): no hay vulnerabilidades conocidas reportadas para esa dependencia y no ve nada más que analizar. **Con RAG, el modelo reconoce el patrón** (cesión de mantenimiento + dependencia nueva sin historial, con versión inicial 0.0.1) como el mismo tipo de estructura que event-stream, coa/rc y colors/faker documentan en el corpus, y sube a backdoor (70). No es el mismo paquete ni el mismo incidente – es **generalización real del patrón**, no memorización del ejemplo exacto.

sandboxed-eval-plugin-system confirma que el RAG no solo sirve para detectar, también para no sobrereaccionar. El código usa eval() con un espacio de nombres vacío de __builtins__ y solo funciones matemáticas permitidas – genuinamente seguro. Sin RAG, el modelo se queda en la superficie ("eval() sobre una expresión de usuario... riesgo inherente de ejecución de código remoto") y marca backdoor (60): un falso positivo. Con RAG, generaliza desde el caso de calibración benign-plugin-loader del corpus y baja a ninguna (5), con una justificación que sí reconoce las mitigaciones concretas del código.

npm-postinstall-worm y build-macro-tampering no muestran diferencia – y eso también es un hallazgo real, no un caso fallido. La hipótesis de partida era que un incidente posterior al corte de entrenamiento (npm-postinstall-worm, patrón "Shai-Hulud", real de 2025) obligaría al modelo a depender del RAG. No fue así: el modelo no necesita saber que el incidente se llama "Shai-Hulud" para reconocer que un script postinstall que descarga un binario externo, lo ejecuta, y usa tokens robados para modificar repositorios de GitHub es malicioso – las señales individuales son autoexplicativas sin necesitar el nombre del caso. Lo mismo pasa con build-macro-tampering (dd ... | xxd -r -p | sh ejecutando shell decodificado desde un fixture): el patrón

es transparente en el propio diff. El RAG importa más cuando la señal maliciosa está distribuida o es sutil (**maintainer-handoff**), no cuando el código en sí ya es inequívoco, por muy reciente o desconocido que sea el incidente que lo inspiró.

fake-checksum-curl-exec es una advertencia, no un fallo – pero merece quedar documentada. Este caso imita a propósito la forma del caso de calibración benign-curl-checksum-verified (que enseña al modelo, vía RAG, que curl + verificación de *checksum* puede ser legítimo) pero el “checksum” se descarga del mismo dominio no confiable que el binario, así que no verifica nada de verdad. Ambas versiones lo marcan correctamente como backdoor – no hay falso negativo. Pero la diferencia en el razonamiento es reveladora:

Sin RAG (score 80, confianza alta): “...permitiendo a un atacante que comprometa dicho dominio servir un binario malicioso validado con un checksum también malicioso...” – identifica sin ayuda que el checksum es autorreferencial.

Con RAG (score 70, confianza media): “...aunque incluye verificación de checksum, podría introducir un binario malicioso si el dominio de origen... está comprometido” – razonamiento más condicional, y el score baja 10 puntos.

El RAG no le hizo fallar el veredicto, pero sí lo hizo **más indulgente** con exactamente el tipo de caso que su propio ejemplo de calibración podría enseñarle a tolerar de más. Es el riesgo estructural de cualquier ejemplo de calibración en el corpus: reduce falsos positivos en el caso que motivó añadirlo (aquí, benign-plugin-loader), pero abre una superficie a que un atacante construya un caso que *parezca* cumplir las condiciones del ejemplo benigno sin cumplirlas de verdad. No cambió el veredicto esta vez – pero con un caso más cuidadosamente construido, podría.

3. Comparativa entre modelos

Mismos casos, ejecutados también contra gemini-3.5-flash y gemini-3.6-flash (con RAG activado, salvo donde se indica). Los tres modelos siguen disponibles y no están marcados como obsoletos en la API a fecha de este informe – verificado contra `models.list()` real, no contra el catálogo estático (dos modelos del catálogo, gemini-2.0-flash y gemini-2.5-flash-lite, devuelven 404 NOT_FOUND al intentar usarlos de verdad pese a aparecer listados).

3.1 Calidad – caso trojan-source (con RAG)

Modelo	risk_score	Categoría	¿Reconoce que el # neutraliza el payload?
gemini-2.5-flash	25	ofuscacion	Sí
gemini-3.5-flash	45	ofuscacion	Sí
gemini-3.6-flash	15	ofuscacion	Sí

Los tres aciertan la categoría y el razonamiento técnico. El modelo más nuevo (3.6-flash) fue el más comedido en severidad, no el más alarmista.

3.2 Calibración – caso benign-plugin-loader (con y sin RAG)

Modelo	Con RAG	Sin RAG	¿Necesita el RAG para calibrar bien?
gemini-2.5-flash	10 (ninguna, correcto)	80 (backdoor, falso positivo)	Sí, mucho
gemini-3.5-flash	5 (ninguna)	5 (ninguna)	No – ya calibrado sin RAG
gemini-3.6-flash	5 (ninguna)	5 (ninguna)	No – ya calibrado sin RAG

Hallazgo principal de esta sección: los modelos de generación 3.x llegan mejor calibrados de fábrica para distinguir “código técnicamente avanzado” de “código malicioso” – no necesitan el RAG para evitar este falso positivo concreto, a diferencia de gemini-2.5-flash. Esto no invalida el RAG (sigue aportando en el reconocimiento de técnicas nombradas, sección 3.1), pero su contribución relativa **depende del modelo**, más crítica cuanto más barato/antiguo es el modelo.

3.3 Coste real por análisis

Precio oficial verificado en <https://ai.google.dev/gemini-api/docs/pricing> (tier estándar, estado en el momento del informe). Tokens de entrada/salida capturados de `usage_metadata` de llamadas reales sobre el caso trojan-source.

Modelo	\$ / 1M tokens entrada	\$ / 1M tokens salida	tokens entrada	tokens salida (real) [†]	coste/análisis	vs. 2.5-flash
gemini-2.5-flash	\$0.30	\$2.50	2724	3248	\$0.0089	1.00x
gemini-3.6-flash	\$1.50	\$7.50	2724	1718	\$0.0170	1.90x
gemini-3.5-flash	\$1.50	\$9.00	2724	2132	\$0.0233	2.60x

[†] **Nota técnica importante:** el campo `candidates_token_count` de la API **no** refleja el gasto real de tokens de salida. Los modelos 2.5+ tienen *tokens de razonamiento (thinking)* internos que se facturan como salida pero no aparecen en ese campo. La cifra correcta es `total_token_count - prompt_token_count`; usar solo `candidates_token_count` habría subestimado el coste real entre un 25 % y un 96 % según el modelo. Dato curioso: gemini-3.6-flash “piensa” con menos tokens que gemini-3.5-flash para el mismo caso (1718 vs. 2132) – más eficiente en razonamiento, aunque el precio por token de toda la generación 3.x es varias veces mayor que el de 2.5-flash.

4. Sobre el catálogo de modelos

Se recibió un catálogo (`docs/GEMINI_MODELS_CATALOG.md`) con 58 modelos. Se verificó contra `GET /v1beta/models` real: **coincide exactamente**, 58 de 58 – no contenía entradas inventadas. Sin embargo, **aparecer en el catálogo no garantiza que el modelo responda**: gemini-2.0-flash y gemini-2.5-flash-lite están listados pero devuelven `404 NOT_FOUND` al invocarlos (el primero, confirmado por la propia documentación de Google, fue retirado el 1 de junio de 2026). Cualquier elección de modelo debe verificarse con una llamada real, no solo con el listado.

5. Conclusiones y recomendación

1. **El RAG no es un “más es mejor” uniforme – y eso no es un problema del RAG, es una consecuencia de qué casos se prueban.** En los 11 casos canónicos (patrones muy conocidos) apenas marca diferencia. En los 5 casos diseñados a propósito para no depender del conocimiento genérico del modelo (sección 2.2), **decide el veredicto en 2 de 5**: detecta un patrón de cesión de mantenimiento malicioso que sin RAG pasa completamente desapercibido (`maintainer-handoff-single-commit`, 25/70), y evita un falso positivo grave sobre código legítimo que usa `eval()` de forma segura (`sandboxed-eval-plugin-system`, 60/5).
2. **El RAG generaliza patrones, no solo repite ejemplos memorizados.** Ninguno de los dos casos anteriores coincide literalmente con ningún documento del corpus – son paquetes e incidentes inventados que comparten la *estructura* de casos reales ya documentados (`event-stream/coa/rc/colors+faker` para el primero, `benign-plugin-loader` para el segundo). Es la prueba más sólida de que el corpus no es solo una lista de incidentes memorizables.
3. **Conocer el nombre de un incidente reciente importa menos de lo esperado cuando el código en sí ya es inequívoco.** El caso `npm-postinstall-worm`, modelado sobre el gusano real “Shai-Hulud” (npm, 2025) – posterior al corte de entrenamiento de los tres modelos probados – se detecta igual de bien con o sin RAG (98 vs. 99), porque las señales del propio diff (descarga y ejecución de un binario externo, uso de tokens robados para modificar repositorios) ya son suficientes sin necesitar reconocer el incidente por su nombre. El RAG aporta más cuando la señal está distribuida o es sutil, no cuando el propio código ya la delata.
4. **Un ejemplo de calibración en el corpus puede crear una superficie de abuso, no solo resolver falsos positivos.** El caso adversarial `fake-checksum-curl-exec` – construido para imitar la forma superficial de `benign-curl-checksum-verified` sin cumplir su condición real – se sigue detectando correctamente con

RAG (70, backdoor), pero con menos confianza y un razonamiento más condicional que sin RAG (80, confianza alta). No falló esta vez, pero es una advertencia real: cada ejemplo benigno añadido al corpus para reducir falsos positivos debe evaluarse también por si un atacante puede construir un caso que aparente cumplir sus condiciones sin cumplirlas.

5. **Los modelos más nuevos reducen la dependencia del RAG para calibración, no la eliminan.** gemini-3.5-flash y gemini-3.6-flash ya no necesitan el RAG para el caso benign-plugin-loader, pero el RAG les sigue aportando contexto útil en el caso trojan-source (reconocimiento de la técnica y su CVE); no se ha repetido aún la batería de casos difíciles de la sección 2.2 contra estos modelos (ver punto 8).
6. **gemini-3.6-flash es la mejor opción si se quiere migrar de 2.5-flash:** mejor calibrado (sección 3.1) y notablemente más barato que 3.5-flash (1.90× vs. 2.60× el coste de 2.5-flash) – aunque sigue siendo casi el doble de caro que el modelo actual en producción.
7. **Recomendación:** mantener gemini-2.5-flash como *default* de coste mínimo y ofrecer gemini-3.6-flash como alternativa configurable (WATCHGATE_LLM_MODEL=gemini-3.6-flash, ya soportado sin cambios de código) para despliegues donde la calibración importe más que el coste por análisis. Migrar el *default* de producción es una decisión de equipo, no solo técnica – casi duplica el coste por PR analizado.
8. **Pendiente:**
 - Investigar por qué ni el RAG ni el cambio de modelo corrigen el falso positivo de benign-curl-checksum-verified – candidato para una entrada de corpus dedicada, siguiendo el mismo método que ya funcionó para trojan-source (explicar la regla técnica exacta, no un recordatorio genérico).
 - Repetir los 5 casos difíciles de la sección 2.2 contra gemini-3.5-flash/gemini-3.6-flash, para saber si los modelos más nuevos también reducen la necesidad de RAG en generalización estructural, o si ahí sigue siendo indispensable independientemente del modelo.
 - Endurecer fake-checksum-curl-exec (o variantes suyas) como caso de regresión permanente, dado que apunta a un punto ciego estructural, no anecdótico, de cualquier corpus con ejemplos de calibración.